

Corporate BlaBla Decoder

Mimari, veri modeli, kod ve kararların tam dökümü. Bu dokümanı devralan bir yazılımcı, projeyi adım adım nasıl inşa ettiğimizi, hangi kararı neden verdiğimizi ve her parçanın nasıl çalıştığını buradan eksiksiz anlayabilir.

Bu doküman ne anlatıyor

Bu, bir devir (handoff) dokümanıdır. Amacı, kod tabanını ilk kez devralan bir yazılımcının; mimari değişmezleri, her modülün sorumluluğunu, kritik kod parçalarını ve en önemlisi **bu kararların neden böyle verildiğini** tek bir yerden anlamasıdır. Ürün vizyonu ve kullanıcı akışı için ayrı **Ürün Dokümanı** vardır; burada odak tamamen teknik.

Teknoloji yığını (özet)

KATMAN	TEKNOLOJİ	ROL
Paylaşılan IP	TypeScript (<code>packages/core</code>)	Tipler, ton tanımları, few-shot kütüphaneleri, prompt builder'lar. Stack-agnostik.
Backend	Firebase Functions v2 (Node 20)	Claude proxy, kota, premium gating, RevenueCat webhook, zamanlanmış işler.
Veri	Firestore + Security Rules	Kullanıcı belgeleri, günlük içerik, agregat sayaçlar. Hassas alanlar sunucu-yazımı.
Mobil	Expo SDK 54 · Expo Router · Reanimated 4	4 sekme, Ton Kadranı, paylaşım kartı, anonim auth, yerel gizlilik.
Abonelik	RevenueCat	Satın alma + entitlement; webhook ile <code>tier</code> 'ı yazan tek kaynak.
İçerik	tsx CLI (<code>content-pipeline</code>)	Günlük Plaza kartlarını Claude ile batch üretip Firestore'a yazar.
Admin	Next.js 15 (App Router)	Sunucu-tarafli Firebase Admin SDK; Plaza editörü + metrikler.
Web	Firebase Hosting	Landing + KVKK/yasal sayfalar (locale-aware).

Dokümanın okunma sırası

Önce **inşa hikâyesini** (hangi sırayla, neden) okuyun — bu, kod tabanının zihinsel haritasını verir. Sonra **mimari değişmez kuralı** içselleştirin; kod tabanındaki hemen her güvenlik kararı oradan türer. Ardından modüller (`core` → `functions` → `firestore` → `mobile` → `pipeline` → `admin`) sırayla, her birinin gerçek kod alıntılılarıyla. En sonda güvenlik tehdit modeli, dağıtım/devir adımları ve "sesi yeniden ayarlama" operasyon kılavuzu var.

İnşa hikâyesi: hangi sırayla ve neden

Proje rastgele bir sırayla yazılmadı. Her adım bir öncekinin üstüne, bilinçli bir bağımlılık mantığıyla kuruldu. Devralan yazılımcının kod tabanını "neden böyle bölünmüş" diye sorgulamadan anlaması için, izlediğimiz yolu olduğu gibi aktarıyorum.

0 Fikir ve sesin sabitlenmesi (kod öncesi)

Beyaz yakalı komütçü analiziyle başladık; viral yakıtın kurumsal frustrasyon olduğuna karar verdik. **Kritik karar:** kod yazmadan önce "ses"i sabitledik — Decoder ve Composer için TR/EN altın few-shot örneklerini elle yazdık. Çünkü ses, stack'ten bağımsız ve kopyalanamaz olan kısım; iskeleti boş kutuya kurmamak için önce o netleşmeliydi.

1 IP çekirdeği — `packages/core`

İlk yazılan kod. Tip sözleşmeleri, ton tanımları, few-shot kütüphaneleri ve iki system-prompt builder. Firebase yok, React Native yok — saf IP. **Neden ilk:** hem proxy hem app bunu tüketecek; tek doğruluk kaynağı burada olmalı.

2 Cloud Function proxy — `functions`

`core`'u tüketen ilk gerçek tüketici. Claude anahtarını tutan, kotayı ve premium gating'i uygulayan tek yer. Mimari değişmez kural burada hayata geçti: anahtar client'ta asla yok.

3 Expo iskeleti — `apps/mobile`

Expo Router, 4 sekme, Reanimated Ton Kadranı, `react-native-view-shot` paylaşım-kartı motoru. Onaylanan kâğıt/mürekkep/damga estetiği `theme.ts` 'te token'a dönüştü. App backend'e yalnızca proxy üzerinden konuşur.

4 Firebase + RevenueCat + bildirimler

Anonim Auth, Firestore şeması + güvenlik kuralları, RevenueCat webhook'u (`tier` 'ı yazan tek kaynak), paywall, ve commute-timed yerel bildirimler.

5 Günlük Plaza pipeline — `content-pipeline`

Günlük kartları canlı API'ye bağlamadık (maliyet + tutarlılık). Claude ile aylık batch üretip `dailyPlaza/{date}` 'e yazan tsx CLI. App sadece günün kartını çeker.

6 Mağaza hazırlığı + Admin

İkonlar (sharp ile), EAS profilleri, locale-aware yasal sayfalar, ASO metadata, submission checklist; ardından Next.js admin paneli (sunucu-tarafli Admin SDK, HMAC şifre kapısı, Plaza editörü).

7 Dört özellik dalgası

Dalga 1: Arşiv + Haftalık Karne + premium savunma. **Dalga 2:** Ton Hafızası (persona — core tipi + prompt enjeksiyonu + sunucu gating). **Dalga 3:** App Check + analytics + crash/ErrorBoundary. **Dalga 4:** Paylaşım kartı QR + erişilebilirlik geçişi.

8 QA turları + isim güncellemesi

Typecheck, syntax, davranış testleri, import çözümleme, string/tema/export denetimleri. Son olarak TR ürün adı her yerde "Corporate BlaBla Decoder" olarak güncellendi (US "Per My Last Email" korundu).

Taşıyıcı ilke: bağımlılık sırası. `core` (kimseye bağlı değil) → `functions` (yalnızca core'a) → `mobile` (core + proxy'ye) → `pipeline/admin` (core + Firestore'a).

Bu sıra, her adımın bir öncekini sağlam bir zemin olarak kullanmasını sağladı.

Mimari ve değişmez kural

Tüm sistem tek bir cümle etrafında kurulur ve devralan herkesin bunu ezbere bilmesi gerekir:

MİMARİ DEĞİŞMEZ KURAL

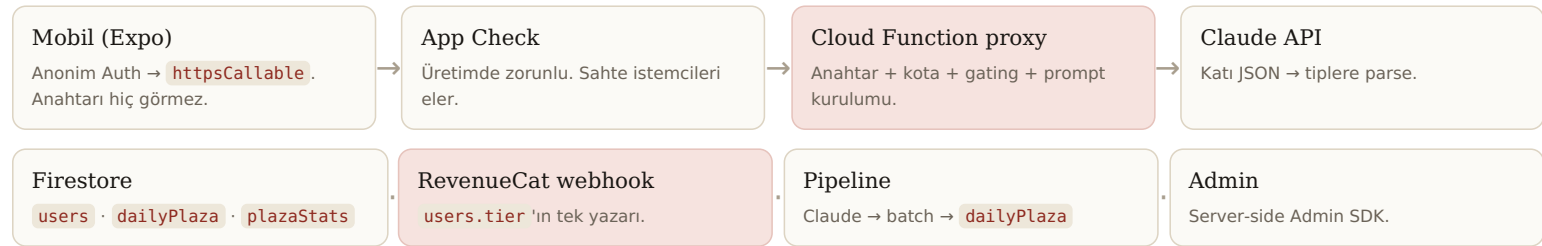
(1) Claude API anahtarı client'ta **asla** bulunmaz — yalnızca Cloud Function proxy'sinde. **(2)** `users.tier` (premium gerçeği) yalnızca RevenueCat webhook'u tarafından yazılır. **(3)** Kota ve premium gating **yalnızca sunucuda** uygulanır; `firestore.rules` istemcinin `tier / usage` 'a dokunmasını engeller.

Tehdit modeli — neden böyle

Mobil istemci düşman varsayılır. Birisi APK'yı tersine çevirebilir, istekleri taklit edebilir, kendi Firestore yazımlarını deneyebilir. Bu üç kural tam da bu senaryolara karşı:

SALDIRI	SAVUNMA
Anahtarı APK'dan çıkarıp doğrudan Claude'u kullanmak	Anahtar client'ta hiç yok; yalnızca proxy secret'ında (<code>defineSecret</code>).
Kendine premium vermek (Firestore'da <code>tier='premium'</code> yazmak)	Kurallar <code>tier</code> 'ı istemci-yazımına kapatır; yalnızca Admin SDK (webhook) yazar.
Günlük kotayı sıfırlamak (<code>usage</code> 'ı silmek)	<code>usage</code> da istemci-yazımına kapalı; sayaç sunucu transaction'ında artar.
Sahte istemciden bedava çağrı yağdırmak	App Check (üretimde zorunlu) + auth zorunluluğu + girdi/uzunluk limitleri.
Premium tonları free'yken almak	Proxy, free kullanıcıda premium ton metnini sunucuda maskeler — kilitli içerik istemciye hiç ulaşmaz.

Veri akışı



Neden anonim auth? Çünkü sürtünme viralitenin düşmanı — hesap zorlaması dönüşümü öldürür. Ama proxy'nin yine de istek başına bir kimliğe ihtiyacı var (kota, tier). Anonim Auth ikisini birden çözer: kullanıcıdan hiçbir şey istemeden Firebase bir `uid` verir, bu `uid` aynı zamanda RevenueCat `appUserID` 'sidir, böylece webhook doğrudan doğru kullanıcı belgesine yazar.

Monorepo ve build topolojisi

npm workspaces kullanıyoruz (kök `package.json` içinde `"workspaces": ["packages/**", "apps/**", "admin"]`). Paylaşılan IP (`packages/core`) hem proxy hem app tarafından import edilir.

```
corporate-blabla/

├── packages/core/          # IP: types, tones, fewshot, prompt builder'lar
│   ├── src/{types,tones,plaza,index}.ts · prompts/{decoder,composer}.ts · fewshot/*.{tr,en}.ts
│   └── functions/          # Firebase v2 Cloud Functions (esbuild ile core'u bundle'lar)
│       ├── src/{config,quota,anthropic,validate,decode,compose,plaza,revenuecat,reengage}.ts
├── apps/mobile/           # Expo SDK 54 · Expo Router
│   ├── app/* · src/{theme,lib/*,components/*} · app.json · eas.json
├── content-pipeline/       # tsx CLI: Claude → dailyPlaza/{date}
├── web/                   # Firebase Hosting: index + privacy/terms (EN) + gizlilik/kullanim (TR)
├── admin/                 # Next.js 15 admin (server-side Admin SDK)
├── firestore.rules · firestore.indexes.json
└── firebase.json          # functions (predeploy build) + hosting (cleanUrls, /kvkk rewrite)
```

Kritik build kararı: functions neden core'u esbuild ile bundle'lar

Firebase yalnızca `functions/` klasörünü zipleyip deploy eder ve orada `npm install` çalıştırır. `packages/core` bu klasörün *dışında* olduğu için, normal bir `import '@corporate-blabla/core'` deploy'da çözülemezdi. Çözüm: predeploy adımıyla esbuild, core'u fonksiyon bundle'ının içine **inline** eder. Ağır SDK'lar (firebase-admin, firebase-functions, @anthropic-ai/sdk) **external** bırakılır (bulutta normal kurulur); yalnızca kendi kodumuz ve core inline edilir. `firebase.json` içindeki `predeploy: ["npm --prefix functions run build"]` bunu tetikler.

Devralırken dikkat: core'da bir değişiklik yaptıysan, functions'ı yeniden deploy etmeden önce `npm --prefix functions run build` çalıştır — aksi halde eski inline core deploy edilir.

packages/core — kopyalanamaz IP

Bu paket ürünün ruhudur ve kasıtlı olarak stack-agnostiktir: Firebase'e, React Native'e, hiçbir runtime'a bağlı değildir. Hem proxy hem app çıktı sözleşmesini buradan paylaşır. Few-shot havuzu da burada — modelin herkeste aynı olduğu bir dünyada, asıl moat budur.

1 • Çıktı sözleşmesi (types.ts)

Modele, bu şekillere **birebir** uyan JSON döndürmesi söylenir; app aynı şekillere karşı parse eder. **Tasarım kararı:** nesne *anahtarları* İngilizcedir (kod için sabit, locale değişse de değişmez); *değerler* kullanıcının diline lokalizedir. Bu, i18n eklemeyi kodun şeklini bozmadan mümkün kılar.

```
packages/core/src/types.ts

export type Locale = 'tr' | 'en';
export type ToneId = 'kind' | 'distant' | 'surgical' | 'boss';
export const TONE_ORDER: ToneId[] = ['kind','distant','surgical','boss']; // kadran sırası

export interface DecoderResult {
  translation: string; // düz, bıçak gibi tercüme
  meter: number; // pasif-agresiflik 0-100
  traps: { phrase: string; meaning: string }[]; // 1-3 tuzak
  defense?: string; // premium "savunma önerisi"
}
export interface ComposerResult {
  intent: string;
  variants: Record<ToneId, string>; // her ton için bir yeniden-yazım
}
export interface ComposePersona { // premium "ton hafızası"
  name: string; kind: 'self' | 'recipient'; samples: string[];
}
```

2 • Tonların kişiliği ve mizah ölçeği (tones.ts)

Dört ton bir **TONES** haritasında tanımlıdır. Her tonun bir **direction** metni vardır — kadranın **kışılığı** burada yaşar ve Composer prompt'una aynen enjekte edilir. Tonu yeniden ayarlamak, kodun başka yerine dokunmadan bu metni değiştirmektir. **FREE_TONES** tek bir yerden türer, böylece UI ve proxy aynı gerçeği paylaşır: **surgical** ve **boss** premium, **kind** ve **distant** ücretsiz.

Metre bandı modelden gelmez — **meter** 'dan istemci-tarafında türetilir. Bu fonksiyonun eşiklerini kaydırmak, tüm uygulamayı topyekûn daha acımasız ya da daha ölçülü yapar. Bu yüzden "ürünün mizah ölçeği" odur:

```
packages/core/src/tones.ts

export function meterBand(meter: number): MeterBand {
  if (meter <= 30) return 'innocent'; // masum: gerçekten nötr
  if (meter <= 60) return 'sneaky'; // sinsî: sessiz bir kenar, hafif baskı
  if (meter <= 85) return 'evidence'; // kanıt-bırakan: "belirttiğim üzere"
  return 'nuclear'; // nükleer: loop'a alma, açık eskalasyon
}
export const FREE_TONES: ToneId[] =
  TONE_LIST.filter((t) => !t.premium).map((t) => t.id); // → ['kind','distant']
```

3 • Few-shot kütüphaneleri (fewshot/*) — asıl moat

Decoder ve Composer için TR ve EN ayrı ayrı, elle küratörlenmiş örnekler. Mevcut sayılar: Decoder TR 10 / EN 9, Composer TR 3 / EN 3 senaryo (her senaryo dört tonu içerir). **En kritik karar:** EN setleri *çeviri değil, transcreation*. "Rica etsem" in İngilizce karşılığı yoktur; onun yerine "per my last email", "for visibility", "bandwidth", "happy to discuss" vardır. İki ayrı küratörlük, iki ayrı moat. Model herkeste aynı; ses bu dosyalarda.

Sesi keskinleştirmek: bu dosyalara örnek ekle/değiştir. Tercümeleri kısa ve bıçak gibi tut; mizah doğruluktan gelmeli, abartıdan değil. Set başına ~10-12 örnek idealdir — bağlamak için yeter, prompt'u şişirmeyecek kadar az.

4 • Decoder prompt'u — katı JSON sözleşmesi (prompts/decoder.ts)

Builder, locale'e göre ses kurallarını, metre kalibrasyonunu ve few-shot örneklerini bir araya getirir; modelden **tek bir katı-JSON nesnesi** döndürmesini ister. Prompt'un kalbi şu (gerçek metinden):

```
buildDecoderSystemPrompt(locale, withDefense) — çıktı kuralları

# Your voice
- Short and scalpel-like. The translation is a diagnosis, not an essay.
- The humor comes from being TOO ACCURATE, never from exaggeration.
- Never moralize, never soften. Write every value in {Turkish|English}.

# The meter (0-100, integer)
0-30 innocent · 31-60 sneaky · 61-85 evidence · 86-100 nuclear
Most real corporate messages land 45-85. Reserve 90+ for genuine escalation.

# Output format — CRITICAL
Return ONLY a single JSON object. No markdown, no fences, no preamble.
{ "translation": string, "meter": int, "traps": [{ "phrase", "meaning" }] }
```

Few-shot örnekleri prompt'un sonuna **INPUT: ... / OUTPUT: {...json...}** biçiminde eklenir. **withDefense=true** (yalnızca premium) olduğunda şemaya bir **defense** alanı ve onu üreten bir talimat eklenir — gating prompt seviyesinde de uygulanır.

5 • Composer prompt'u — dört ton tek çağrı + persona (prompts/composer.ts)

Tasarım kararı: Composer dört tonu *tek çağrıda* üretir. Böylece kullanıcı kadranı kaydırırken yeni bir istek atılmaz — sıfır gecikme, sıfır ekstra maliyet. Ton

hafızası (persona) verildiğinde, prompt'a bir "tone memory" bloğu enjekte edilir; örnekler asla aynen kopyalanmaz, yalnızca dört tonun register'ını modüle eder:

packages/core/src/prompts/composer.ts – personaBlock

```
function personaBlock(persona: ComposePersona, locale: Locale): string {
  const samples = persona.samples.map((s) => ` • ${s}`).join('\n');
  return `
# Personalization (tone memory)
The user trained "${persona.name}". Here is how that person writes:
${samples}
Subtly adapt the register, formality, directness and vocabulary of ALL
FOUR variants – but NEVER copy the samples verbatim.`;
}
```

functions — Cloud Function proxy

Firebase Functions v2, Node 20, bölge `europa-west1` (İstanbul'a yakın, düşük gecikme). App'in çağırdığı tek backend yüzeyi budur. Tüm callable'lar `onCall` 'dır — auth doğrulaması yerleşik gelir, böylece her fonksiyonda `req.auth?.uid` hazırır.

Yapılandırma (`config.ts`) — tek dokunuş noktası

Tüm operasyonel sabitler tek dosyada toplanır; secret'lar `defineSecret` ile tanımlanır ve **asla** kaynağa gömülmez.

```
functions/src/config.ts

export const ANTHROPIC_API_KEY = defineSecret('ANTHROPIC_API_KEY'); // client'ta asla yok
export const REVENUECAT_WEBHOOK_AUTH = defineSecret('REVENUECAT_WEBHOOK_AUTH');
export const MODEL = process.env.CLAUDE_MODEL || 'claude-sonnet-4-6'; // ses için Sonnet; maliyet için Haiku'ya düşür
export const FREE_LIMITS = { decode: 3, compose: 3 } as const; // premium sınırsız
export const MAX_INPUT = 2000; // maliyet + suistimal sınırı
export const REGION = process.env.FUNCTIONS_REGION || 'europa-west1';
export const ENFORCE_APP_CHECK = process.env.ENFORCE_APP_CHECK === 'true'; // dev build hazır olunca true
```

compose handler — auth, doğrulama, kota, gating, maskeleye

Akış her zaman aynı sırada: kimlik → girdi doğrulama → kota (sunucu) → persona sanitizasyonu (premium-gate) → prompt → Claude → doğrula → free kullanıcıda premium tonları maskeleye. Premium ton metni sunucuda boşaltılır; kilitli içerik istemciye **hiç ulaşmaz** ama kullanıcı tonların var olduğunu görür (dönüşüm için).

```
functions/src/compose.ts (özet)

export const compose = onCall(
  { region: REGION, secrets: [ANTHROPIC_API_KEY], memory: '256MiB',
    timeoutSeconds: 30, enforceAppCheck: ENFORCE_APP_CHECK },
  async (req) => {
    const uid = req.auth?.uid;
    if (!uid) throw new HttpsError('unauthenticated', 'Sign-in required.');
```

```
    const { intent, locale, persona } = (req.data ?? {}) as ComposeRequest;
    if (typeof intent !== 'string' || !intent.trim())
      throw new HttpsError('invalid-argument', 'intent is required.');
```

```
    if (intent.length > MAX_INPUT) throw new HttpsError('invalid-argument', '...');
```

```
    const { tier, remaining } = await consumeQuota(uid, 'compose'); // kotanın TEK yeri
    const safePersona = sanitizePersona(persona, tier === 'premium');// ton hafızası premium-only
```

```
    const system = buildComposerSystemPrompt(locale, safePersona);
    const result = await completeJSON<ComposerResult>(client, { system, user: intent.trim(), maxTokens: 800 });
    validateComposer(result);
```

```
    const locked = tier === 'free' ? PREMIUM_TONES : [];
    for (const tone of locked) result.variants[tone] = ''; // kilitli ton metnini sunucuda boşalt
    return { result, meta: { tier, remaining, locked } };
  }
);
```

`sanitizePersona` ayrıca suistimali sınırlar: yalnızca premium kullanıcıda çalışır, en fazla 5 örnek × 500 karakter alır, isim 60 karaktere kapanır. `decode` handler'ı aynı iskelete sahiptir; tek farkı `withDefense` 'in yalnızca premium'da onurlandırılması (`allowDefense = tier === 'premium' && withDefense === true`) ve free kullanıcıda `defense` alanının silinmesidir.

Kota — yarış-koşulsuz transaction (`quota.ts`)

Kota bir Firestore transaction'ında atomik olarak okunur ve artırılır; iki eşzamanlı istek limiti aşamaz. Gün, `RESET_TZ` (varsayılan UTC) zaman diliminde `YYYY-MM-DD` anahtarıyla hesaplanır; tarih değişince sayaç sıfırlanır. `tier` burada *okunur* ama yazılmaz (onu webhook yazar).

```
functions/src/quota.ts (çekirdek)

return db.runTransaction(async (tx) => {
  const data = (await tx.get(ref)).data() ?? {};
  const tier = data.tier === 'premium' ? 'premium' : 'free'; // eksik doc → free
  const usage = data.usage?.date === day ? { ...data.usage } : { date: day, decode: 0, compose: 0 };
  if (tier === 'free' && (usage[action] ?? 0) >= FREE_LIMITS[action])
    throw new HttpsError('resource-exhausted', `Daily free limit reached for ${action}.`);
  usage[action] = (usage[action] ?? 0) + 1;
  tx.set(ref, { tier, usage, lastSeen: FieldValue.serverTimestamp() }, { merge: true });
  return { tier, remaining: /* free: kalan, premium: null */ };
});
```

Claude çağrısı — güvenilir JSON (`anthropic.ts`)

Model'den yapılandırılmış çıktı almanın iki numarası: (1) asistan turunu `{` ile **prefill** etmek — model bir JSON nesnesi ve başka hiçbir şey üretmeye zorlanır; (2) ilk parse başarısız olursa düşük sıcaklıkta bir kez retry. `parseLoose` kod-çitlerini soyar ve en dıştaki süslü parantezlere keser.

```
functions/src/anthropic.ts – completeJSON
```

```
export async function completeJSON<T>(client, { system, user, maxTokens, temperature = 0.8 }): Promise<T> {
  const primary = await client.messages.create({
    model: MODEL, max_tokens: maxTokens, temperature, system,
    messages: [ { role: 'user', content: user },
      { role: 'assistant', content: '{}' } ],    // ~ prefill
  });
  try { return parseLoose<T>('{}' + textOf(primary)); }
  catch {
    const retry = await client.messages.create({ /* temperature: 0.2, "Return ONLY the JSON object." */ });
    return parseLoose<T>('{}' + textOf(retry));
  }
}
```

RevenueCat webhook — `users.tier` 'ın tek yazarı (`revenuecat.ts`)

Bu `onRequest` fonksiyonu, sistemde `tier` 'a yazan **tek** yerdir. RevenueCat'i buraya `Authorization` başlığı `REVENUECAT_WEBHOOK_AUTH` secret'ına eşit olacak şekilde POST atmaya yapılandırırısın; app'in RevenueCat `appUserID` 'sini Firebase `uid` 'ine eşitlersin, böylece `app_user_id` doğrudan kullanıcı belgesine map'lenir.

`functions/src/revenuecat.ts` (çekirdek)

```
const ACTIVE = new Set(['INITIAL_PURCHASE','RENEWAL','UNCANCELLATION',
  'PRODUCT_CHANGE','NON_RENEWING_PURCHASE','SUBSCRIPTION_EXTENDED']);
const INACTIVE = new Set(['EXPIRATION','SUBSCRIPTION_PAUSED']); // iptalde değil, SÜRESİ DOLUNCA düşür

export const revenuecatWebhook = onRequest({ region: REGION, secrets: [REVENUECAT_WEBHOOK_AUTH] },
  async (req, res) => {
    if (req.headers.authorization !== REVENUECAT_WEBHOOK_AUTH.value())
      return void res.status(401).send('unauthorized');
    const e = req.body?.event;
    if (ACTIVE.has(e?.type)) await setTier(e.app_user_id, 'premium', e.expiration_at_ms);
    else if (INACTIVE.has(e?.type)) await setTier(e.app_user_id, 'free');
    res.status(200).send('ok');
  });
```

İptal anında erişim hemen kesilmez — kullanıcı dönem sonuna kadar premium kalır, yalnızca `EXPIRATION` 'da düşürülür. `premiumUntil` ayrıca saklanır ve bir savunma katmanı olarak kullanılabilir (webhook gecikse bile zaman damgası gelecekteyse premium korunur). Diğer iki fonksiyon: `openPlaza` (günün kartını döner, `plazaStats` agregat sayacını artırır) ve `reengageLapsed` (günlük 17:30 UTC; 3+ gündür açmayan, push token'ı olan kullanıcılara Expo Push ile yeniden-etkileşim bildirimi).

Firestore — şema, kurallar, indeksler

Şema

YOL / ALAN	YAZAN	NOTLAR
<code>users/{uid}</code> · <code>tier</code> , <code>premiumUntil</code>	sunucu (webhook)	Entitlement'in tek doğruluk kaynağı. Proxy buna güvenir.
<code>users/{uid}</code> · <code>usage</code> , <code>lastSeen</code>	sunucu (proxy)	Günlük kota sayaçları + son görülme.
<code>users/{uid}</code> · <code>prefs</code> , <code>streak</code> , <code>pushToken</code> , <code>onboardingDone</code> , <code>displayLocale</code>	istemci	Yalnızca güvenli alanlar (aşağıdaki kural whitelist'i).
<code>dailyPlaza/{YYYY-MM-DD}</code>	pipeline	<code>{ date, format, locales: { tr, en } }</code> . Tüm girişli kullanıcılar okur.
<code>plazaStats/{YYYY-MM-DD}</code>	sunucu	"Yalnız değilsin" agregat sayacı.

Güvenlik kuralları — neden böyle (`firestore.rules`)

İstemcinin kendi belgesine yazabileceği alanlar bir whitelist (`safeUserKeys`) ile sınırlıdır. **Kritik:** `tier` ve `usage` bu listede **yoktur** — yalnızca kuralları bypass eden Admin SDK (sunucu) onları yazabilir. Kullanıcının kendine premium vermesini ya da kotasını sıfırlamasını engelleyen tek şey budur. `diff().affectedKeys().hasOnly(...)` kalıbı, update'te yalnızca whitelist alanlarının değiştiğini garanti eder.

```
firestore.rules

function safeUserKeys() {
  return ['prefs','pushToken','onboardingDone','displayLocale','streak','updatedAt'];
}
match /users/{uid} {
  allow read:   if request.auth != null && request.auth.uid == uid;
  allow create: if request.auth.uid == uid
    && request.resource.data.keys().hasOnly(safeUserKeys());
  allow update: if request.auth.uid == uid
    && request.resource.data.diff(resource.data).affectedKeys().hasOnly(safeUserKeys());
  allow delete: if request.auth.uid == uid;           // KVKK: kullanıcı kendini siler
}
match /dailyPlaza/{day} { allow read: if request.auth != null; allow write: if false; }
match /plazaStats/{day} { allow read: if request.auth != null; allow write: if false; }
match /{document=**} { allow read, write: if false; }  // default deny
```

`dailyPlaza` ve `plazaStats` istemciye salt-okunur, yazımı tamamen sunucuya bırakılmıştır. En sonda bir "default deny" kuralı, tanımlanmamış her yolu kapatır. Bileşik sorgu gerektiren tek erişim `reengageLapsed` 'in `lastSeen` filtresidir; ilgili indeks `firestore.indexes.json` 'da tanımlıdır.

apps/mobile — Expo SDK 54

Expo Router (dosya-tabanlı yönlendirme), Firebase JS SDK, Reanimated 4. **Tasarım kararı:** native Firebase yerine *JS SDK* seçildi, çünkü Anonymous Auth ve proxy çağrıları Expo Go'da bile (native config olmadan) çalışsın istedik — geliştirme döngüsü hızlansın. Native gereksinim yalnızca paylaşım-capture ve RevenueCat için var (dev build).

apps/mobile yapısı

```
app/
  layout.tsx          # font yükleme + ErrorBoundary + anonim giriş + initCrash + track('app_open')
  (tabs)/{index,decoder,composer,profile}.tsx  # Günlük / Decoder / Composer / Profil
onboarding · payroll · archive · report · personas  # modal/stack ekranlar
src/
  theme.ts            # marka token'ları (kod olarak tasarım)
  lib/{firebase,api,user,history,personas,notifications,purchases,analytics,crash,locale,strings}.ts
  components/{ToneDial,ShareCard,Meter,PlazaCard,Header,ErrorBoundary,ui/*}.tsx
```

Tasarım = kod (`theme.ts`)

Onaylanan estetik doğrudan token'a çevrildi. Light ve dark iki ayrı palet; ama `paperTheme` her zaman kâğıttır ve `ShareCard` bunu kullanır — paylaşım kartının grup sohbetinde tanınması bu kâğıt+damga kimliğine bağlı.

apps/mobile/src/theme.ts (özet)

```
export const STAMP_RED = '#BC3B2E';
const light = { mode:'light', bg:'#F4F1EA'/*kâğıt*/, card:'#FBFAF6', text:'#221F1B'/*mürekkep*/,
  accent:STAMP_RED, /* ... */ };
const dark = { mode:'dark', bg:'#1B1916'/*fûme*/, card:'#23201C', text:'#F0EBE0'/*krem*/,
  accent:'#D6584A', /* ... */ };
export const paperTheme = light; // ShareCard her iki temada da kâğıt kalır
export const radius = { sm:6, md:9, lg:12, xl:16, pill:999 };
export const space = { xs:4, sm:8, md:12, lg:16, xl:22, xxl:28 };
export const font = { serif:'Fraunces_600SemiBold', sans:'Inter_400Regular', /* ... */ };
```

İmza etkileşim — Ton Kadranı (`components/ToneDial.tsx`)

Reanimated 4 + Gesture Handler. Parmak sürüklenirken `progress` shared value canlı takip eder; bırakınca en yakın tona `withSpring` ile snap'ler ve `runOnJS` ile JS tarafında haptik + çıktı güncellemesi tetikler. Tap ile de seçilebilir (`Gesture.Exclusive`). Erişilebilirlik için

`accessibilityRole="adjustable"`.

components/ToneDial.tsx (çekirdek)

```
const pan = Gesture.Pan()
  .onUpdate((e) => { progress.value = clamp(start + e.translationX / step, 0, 3); })
  .onEnd(() => {
    const i = Math.round(progress.value);
    progress.value = withSpring(i); // en yakın tona snap
    runOnJS(commit)(i); // haptik tıkırtı + çıktıyı değiştir
  });
const gesture = Gesture.Exclusive(pan, Gesture.Tap().onEnd(/* tap-to-select */));
```

Viral artefakt — ShareCard (`components/ShareCard.tsx`)

Kart `forwardRef` ile bir `View` ref'i dışarı verir; `captureRef` bunu PNG'ye basar ve `expo-sharing` OS paylaşım sayfasına gönderir. Kart `paperTheme` ile her iki modda da kâğıt kalır; karekod `react-native-qrcode-svg` ile gömülür.

components/ShareCard.tsx (capture)

```
export const ShareCard = forwardRef<View, ShareCardProps>(function ShareCard(props, ref) { /* ... */ });

export async function shareCard(ref) {
  const uri = await captureRef(ref, { format: 'png', quality: 1 });
  if (await Sharing.isAvailableAsync()) await Sharing.shareAsync(uri, { mimeType: 'image/png' });
}
```

Durum ve API

Kullanıcı bağlamı (`lib/user.tsx`): anonim giriş yapar, Firestore kullanıcı belgesine abone olur ve `{ uid, loaded, tier, prefs, streak, onboardingDone, locale, updatePrefs, completeOnboarding, markDailyEngaged }` sözleşmesini verir. İstemci yalnızca güvenli alanları yazar (kurallarla uyumlu). **API istemcisi (`lib/api.ts`):** proxy'ye tipli sarmalayıcılar (`decode/compose/openPlaza`); `resource-exhausted` hatasını lokalize bir kota mesajına çevirir.

apps/mobile/src/lib/api.ts

```
export async function compose(intent: string, locale: Locale, persona?: ComposePersona) {
  const fn = httpsCallable<unknown, ComposeResponse>(functions, 'compose');
  const { data } = await fn({ intent, locale, persona });
  return data; // { result, meta: { tier, remaining, locked } }
}
```

Gizlilik: hassas veri yalnızca cihazda

Kullanıcının gerçek mailleri (Arşiv) ve eğittiği personalar (Ton Hafızası) sunucuya **hiç** gitmez — AsyncStorage'da, cihazda saklanır. Sunucu yalnızca o anki çağırıyor görür, profil olarak tutmaz. Bu, ürünün gizlilik vaadinin kod-seviyesindeki garantisidir.

```
const KEY = 'cb_history_v1'; const MAX = 120;
// History lives ONLY on the device (AsyncStorage), never on our servers.
export async function getHistory(): Promise<HistoryEntry[]> {
  const raw = await AsyncStorage.getItem(KEY);
  return raw ? JSON.parse(raw) : [];
}
export async function addHistory(entry: Omit<HistoryEntry, 'id'|'ts'>) { /* prepend + slice(0,MAX) */ }
```

Bildirimler (`lib/notifications.ts`): commute-timed yerel hatırlatmalar cihazda zamanlanır (sunucu zaman-dilimi cebelleşmesi olmadan kesin pencere). İzin akışı + Expo push token kaydı da burada. **Sağlamlık dikişleri**: `analytics.ts` (7 yüzeyde `track()` , PostHog seam), `crash.ts` + `ErrorBoundary` (Sentry seam) — ikisi de tek satırla bağlanır. Fontlar `_layout.tsx` 'te `expo-font` ile yüklenir; yüklenmezse sistem fontuna düşer.

Expo Go vs dev build: Navigasyon, Ton Kadranı ve Firebase çağrıları Expo Go'da çalışır. Paylaşım-capture ve RevenueCat **dev build** ister. `package.json` 'daki native sürümleri `npm expo install` ile SDK 54'e tam hizala.

content-pipeline ve admin

content-pipeline — Günlük Plaza üretimi

tsx CLI. Günlük kartları canlı API'ye bağlamak yerine Claude ile aylık batch üretilip `dailyPlaza/{date}` 'e yazar. **Neden batch:** maliyet öngörülebilirliği ve içerik tutarlılığı — günde binlerce kullanıcı aynı kartı çekecekse, onu bir kez üretmek mantıklı. Voice rules + format rotasyonu (`trap → riddle → horoscope → bingo`) prompt'larda. Hem pipeline hem app, bir payload'a güvenmeden önce core'dan paylaşılan `isValidPayload` ile doğrular:

```
packages/core/src/plaza.ts — isValidPayload

export function isValidPayload(format: PlazaFormat, p: any): boolean {
  switch (format) {
    case 'trap':      return typeof p.incoming === 'string' && typeof p.reading === 'string'
                      && typeof p.meter === 'number' && p.meter >= 0 && p.meter <= 100;
    case 'bingo':     return Array.isArray(p.cells) && p.cells.length === 9;    // 3x3
    case 'horoscope': return typeof p.sign === 'string' && typeof p.text === 'string';
    case 'riddle':    return typeof p.term === 'string' && typeof p.answer === 'string';
  }
}
```

admin — Next.js 15 (sunucu-terafli Admin SDK)

Tüm veri erişimi **server component / server action** içinde Firebase Admin SDK ile yapılır — anahtarlar tarayıcıya hiç gitmez. Erişim, HMAC imzalı bir cookie ile korunur. `verifySession` sabit-zamanlı karşılaştırma (`timingSafeEqual`) kullanır; `requireAdmin` her korumalı sayfanın başında çağrılır ve geçersizse `/login` 'e yönlendirir.

```
admin/lib/auth.ts (çekirdek)

export function signSession(payload = 'admin'): string {
  const sig = crypto.createHmac('sha256', SECRET).update(payload).digest('hex');
  return `${payload}.${sig}`;
}
export function verifySession(token?: string): boolean {
  const [p, sig] = (token ?? '').split('.');
  const expected = crypto.createHmac('sha256', SECRET).update(p).digest('hex');
  return crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected));
}
export async function requireAdmin() {
  // her korumalı sayfada
  if (!verifySession(await cookies()).get(COOKIE)?.value) redirect('/login');
}
```

Panel üç bölüm sunar: **dashboard** (temel metrikler), **plaza** (günlük kartları listeler + editör — kaydetmeden önce `isValidPayload` ile doğrular), **users**. Plaza kaydı bir server action'dır: `requireAdmin()` → doğrulama → `adminDb.collection('dailyPlaza').doc(day).set(...)` → `revalidatePath('/plaza')`.

web — landing + KVKK

Firebase Hosting (`cleanUrls`). EN için `privacy.html / terms.html`, TR için `gizlilik.html` (KVKK aydınlatma metni) + `kullanim.html`. `firebase.json` 'da `/kvkk → /gizlilik.html` rewrite'ı var. Paylaşım kartındaki karekodun işaret ettiği landing burada.

Güvenlik, dağıtım, devir ve kalite

Gizlilik & KVKK

- Minimum veri:** anonim başlangıç, kişisel veri istenmez; kim olduğun bilinmez.
- Yerel-only hassas veri:** gerçek mailler ve personalar cihazda (AsyncStorage), sunucuda profil tutulmaz.
- Silme hakkı:** kullanıcı kendi belgesini tek dokunuşla siler; kurallar bunu garanti eder (yalnızca kendi belgesi).
- Locale-aware yasal metinler:** TR aydınlatma metni + EN privacy/terms. Yasal metinler bir KVKK avukatınca gözden geçirilmeli.

Dağıtım ve devir adımları

Bunlar devralan yazılımcının yapması gereken, kod-tamam ama yapılandırma-bekleyen adımlardır:

ADIM	YAPILACAK
Secret'lar	<code>firebase functions:secrets:set ANTHROPIC_API_KEY</code> ve <code>REVENUECAT_WEBHOOK_AUTH</code> . Admin için <code>ADMIN_SESSION_SECRET</code> .
Firebase config	Mobil <code>.env</code> (web config), <code>app.json</code> EAS <code>projectId</code> , Apple/Play kimlikleri (<code>eas.json</code> 'daki <code>REPLACE_*</code>).
Deploy	<code>firebase deploy</code> (functions + firestore.rules + indexes + hosting). Functions predeploy esbuild bundle'ı çalıştırır.
RevenueCat	'premium' entitlement + offering oluştur; webhook'u proxy URL'ine, <code>Authorization</code> =secret ile bağla; <code>appUserID</code> =Firebase uid.
İçerik	Pipeline'ı çalıştırıp ilk ay(lar)ın Günlük Plaza kartlarını üret.
EAS build	<code>npx expo install</code> ile native sürümleri SDK54'e hizala; <code>development</code> profili ile dev-build smoke testi.
İsteğe bağlı	PostHog / Sentry / App Check tek-satır bağla (<code>docs/infrastructure.md</code>); <code>ENFORCE_APP_CHECK=true</code> yap.

Dev-build smoke testi (kabul kriteri)

Sandbox'ta tam Expo/Next runtime'ı çalıştırılmaz (native modüller + cihaz gerekir). Statik olarak doğrulanabilecek her şey doğrulandı; geriye cihazda şu sekiz-adımlı akışın doğrulanması kalıyor: **(1)** anonim giriş → **(2)** Günlük Plaza yüklenir → **(3)** Decoder/Composer kotası sayar → **(4)** Ton Kadranı + haptik → **(5)** sandbox satın alma → **(6)** webhook `tier` 'ı premium yapar → **(7)** paylaşım kartı PNG'ye basılır → **(8)** hesap silme.

Kalite — doğrulanmış durum

KONTROL	SONUÇ
Typecheck — core / functions (+esbuild bundle) / pipeline	✓ temiz
Syntax — mobil 30 dosya · admin 14 dosya	✓ 0 hata
Core davranış testi — few-shot örnekleri	✓ 25 doğrulandı
Yerel import çözümleme (mobil)	✓ 177/177
String anahtarları · tema token'ları · core export'ları	✓ 87 · 14 · 17, eksik yok
Metre bantları · ton sırası · payload doğrulama · persona enjeksiyonu	✓ geçti

Operasyon: sesi yeniden ayarlama

Ürünün ruhu koddan ayrılabilir; bir geliştirici stack'e dokunmadan tonu değiştirebilir. Üç kaldıraç:

- Topyekûn daha acımasız/ölçülü:** `meterBand()` eşiklerini kaydır (`tones.ts`).
- Bir tonun karakterini değiştir:** ilgili tonun `direction` metnini düzenle — aynen prompt'a enjekte edilir.
- Sesi keskinleştir:** `fewshot/` dizinine örnek ekle/değiştir. Yeni dil eklemek için `Locale` 'i genişlet, her tona `label` / `direction` ekle, eşleşen few-shot dosyalarını oluştur.

Ürün vizyonu, doğuş hikâyesi ve kullanıcı akışı için ayrı **Ürün Dokümanı**'na bakınız. Bu doküman kod tabanının teknik devir kılavuzudur.